



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

FPGA 课程设计报告

实验 00：基于 Verilog 的 UART 图像接收与 Sobel 边缘 检测 RTL 仿真报告

专 业： 微电子科学与工程

班 级： 微电子2班

队 名： 快跑通

指导老师： 周贤中



集成电路学院

2026 年 6 月 18 日

1. 实验目标与验收要求

- 1. 说明 128 x 72 RGB888 图像如何以字节流方式输入。
- 2. 说明 UART 帧头、行头与像素数据格式。
- 3. 说明 rgb_to_gray 的灰度转换方法，以及 sobel_core 产生边缘强度的过程。
- 4. 提交输入图、Sobel 输出图和包含关键握手信号的波形截图。
- 5. 根据波形判断一帧图像处理是否完成。

2. 实验环境与文件组成

类别	文件	作用
顶层	sobel_system.v	连接 UART 接收、帧解析、灰度转换、Sobel 核心和视频流模型。
接收	uart_rx.v	按设定时钟和波特率接收串口数据，输出 rx_valid/rx_data。
帧解析	image_frame_rx.v	识别帧头、行头和 RGB888 像素，产生 rgb_valid 及像素坐标。
灰度	rgb_to_gray.v	用定点系数把 R/G/B 转换为 8 位灰度。
边缘	sobel_core.v	用两行缓存和 3 x 3 窗口计算 Sobel 边缘强度。
结果	input_rgb.png / sobel_out.png	输入图像预览与仿真生成的边缘输出。

```

● PS C:\Users\Desktop\aa> .\run_sim.ps1
VCD info: dumpfile build/sobel_system_tb.vcd opened for output.
Sobel RGB888 simulation passed
Output image: build/sobel_out.pgm
Waveform: build/sobel_system_tb.vcd
Wrote build/input_rgb.png
Wrote build/sobel_out.png
Simulation finished.
Output image: build/sobel_out.png
Waveform: build/sobel_system_tb.vcd
○ PS C:\Users\Desktop\aa> 

```

图 1 Windows PowerShell 仿真运行结果截图

图 1 中可以看到脚本运行后生成 VCD 波形文件、输入图和 Sobel 输出图，并打印 “Sobel RGB888 simulation passed”，说明测试平台已完成一次完整仿真。

3. UART 图像帧格式

仿真输入不是直接把像素送入 Sobel 核心，而是先模拟成 UART 字节流。该设计与后续上板实验保持同一帧协议，便于从仿真迁移到硬件验证。

```

frame header: 55 aa width_l width_h height_l height_h 18
line header : 33 cc row_l row_h
pixels      : R G B, repeated 128 times per row

```

其中 0x18 表示 RGB888；width 和 height 使用低字节在前的 16 位字段。本实验图像尺寸为 128 x 72，每行 128 个像素，每个像素 3 字节。

4. 系统数据流与模块功能

阶段	输入/输出	关键模块	说明
字节接收	uart_rx_i -> rx_data	uart_rx	空闲检测起始位，按 CLKS_PER_BIT 采样 8 位数 据并在停止位后拉高 rx_valid。
帧解析	rx_data -> RGB + 坐标	image_frame_rx	状态机识别 55 aa 帧头、

			33 cc 行头，并输出 r/g/b、x/y。
灰度转换	RGB -> gray	rgb_to_gray	使用 $77R + 150G + 29B$ 后右移 8 位，近似 $0.299R + 0.587G + 0.114B$ 。
边缘检测	gray -> edge_data	sobel_core	两行缓存构造 3×3 窗口，计算 G_x 、 G_y ，并用 $ G_x + G_y $ 得到边缘强度。
视频输出	edge_data -> RGB 灰度流	video_stream_model	把 Sobel 强度复制到 RGB 三通道，形成可保存和观察的输出图。

4.1 灰度转换实现

rgb_to_gray.v 使用移位友好的定点系数实现灰度化：

```
gray_calc = r * 77 + g * 150 + b * 29
gray      = gray_calc[15:8]
```

77、150、29 的和为 256，因此取高 8 位相当于除以 256，硬件上比浮点计算更容易综合。模块同时延迟传递 x/y 坐标，使后续能知道每个灰度像素的位置。

4.2 Sobel 核心实现

sobel_core.v 使用 line0 和 line1 作为两行缓存，并维护 top/mid/bot 三列移位寄存器，形成 3×3 窗口。核心计算如下：

```
Gx = -top0 + top2 - 2*mid0 + 2*mid2 - bot0 + bot2
Gy = -top0 - 2*top1 - top2 + bot0 + 2*bot1 + bot2
mag = abs(Gx) + abs(Gy)
edge_data = mag > 255 ? 255 : mag[7:0]
```

边界像素因缺少完整邻域而输出 0。帧末尾通过 flush_active 补齐底边与右边输出，并在最后一个输出完成后拉高 edge_frame_done。

5. 仿真过程与关键波形

仿真先由脚本生成输入数据，再调用 iverilog/vvp 编译并运行 testbench。完成后 build 目录中应包含 input_rgb.png、sobel_out.pgm、sobel_out.png 和 sobel_system_tb.vcd。

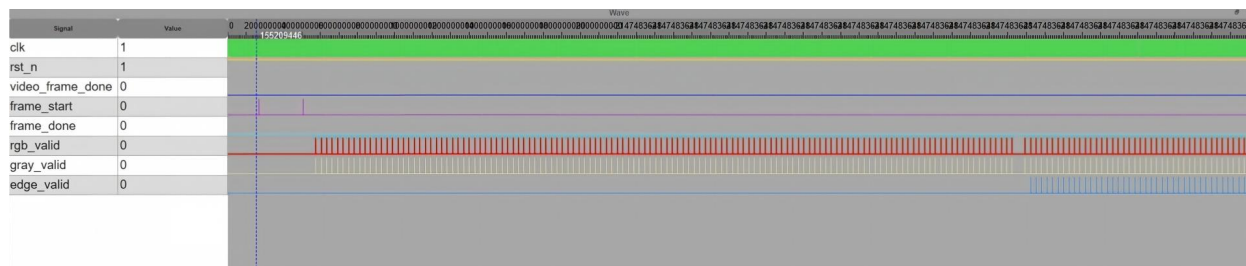


图 3 关键波形截图：rgb_valid、gray_valid、edge_valid 等信号（本地提供图片）

图 3 中 frame_start 在帧头解析完成后出现脉冲；rgb_valid 表示帧解析模块输出有效 RGB 像素；gray_valid 随灰度转换延迟后有效；edge_valid 在 Sobel 窗口满足计算条件后持续出现。最后 edge_frame_done 或 video_frame_done 可作为一帧处理完成的判断依据。

6. 图像结果与现象分析



图 4 输入 图像



图 5 Sobel 边缘检测输出图

从图 4 到图 5 可以看到，Sobel 输出主要保留亮度突变位置：中心垂直线、斜边和矩形边缘被增强，渐变平坦区域则明显变暗。这与 Sobel 算子检测一阶梯度的预期一致。

图中边缘亮度不是简单二值化结果，而是 $|G_x| + |G_y|$ 的饱和值；因此强边缘接近白色，弱边缘和背景纹理表现为较暗的灰度。

7. 可选扩展：更换输入图片并重新仿真

参考实验说明要求至少完成一项基础扩展。本次扩展选择“更换输入图片并重新仿真”，并结合本地文件夹中的 input_rgb.png、sobel_out.png、sobel_out-1.png、sobel_out-2.png 以及扩展波形截图，对输入变化、输出边缘和时序响应进行更完整的分析。

7.1 扩展目的与修改思路

基础实验已经证明了默认图像能够通过 UART 字节流进入 RTL 系统并得到 Sobel 输出。扩展实验的目的不是修改算法结构，而是更换输入图像内容，观察同一条 RTL 数据链在不同边缘形状下的响应是否合理。这样可以进一步验证坐标传递、行缓存窗口扫描和边界处理逻辑。

扩展项	本地证据文件	观察重点
更换输入图	input_rgb.png	灰度渐变背景、绿色竖线、黄色圆环是否能形成可分辨边缘。
重新生成输出	sobel_out.png	圆环边缘和竖线边缘是否在 Sobel 输出中被突出。
重复输出对比	sobel_out-1.png / sobel_out-2.png	多次保存的输出是否保持主要结构一致，是否存在边界差异。
扩展波形观察	f3f6f41149a57971b596615f9e82584f.png	frame_err、video_valid、line_index 等信号是否符合预期。

7.2 扩展输入图像特征



图 6 扩展输入图：渐变背景、绿色竖线和黄色圆环

扩展输入图仍保持 128 x 72 RGB888 尺寸，满足实验帧格式对 width 和 height 的要求。图像中包含三类典型区域：第一类是从左到右变化的灰度背景，用来观察缓慢亮度变化是否会产生弱边缘；第二类是左侧绿色竖线，它在水平方向上产生明显颜色突变；第三类是黄色圆环，其内外边界都存在亮度和颜色变化，适合观察 Sobel 对曲线边缘的响应。

7.3 扩展输出结果对比



图 7 扩展 Sobel 输出



图 8 扩展 Sobel 输出



图 9 扩展 Sobel 输出

从图 7 至图 9 可以看到，绿色竖线在输出中表现为明亮的垂直边缘，黄色圆环则形成近似圆形的双边界轮廓。这说明 `rgb_to_gray` 转换后，颜色突变已经转化为灰度突变，`sobel_core` 的 3×3 窗口能够正确捕捉这些局部梯度。

三个输出文件的主体结构基本一致：竖线边缘位置稳定，圆环边缘集中在原图圆周附近。局部差异主要体现在圆环右侧或边界亮度的连续性上，这类差异可从输出保存时刻、边界补齐和窗口尚未完全覆盖的像素位置理解；但整体边缘分布没有偏移，说明帧解析得到的 x/y 坐标与后级 Sobel 输出坐标保持一致。

渐变背景不会形成大面积高亮区域，说明 Sobel 对缓慢变化区域响应较弱。

竖线边缘亮度高且位置明确，说明水平梯度计算 G_x 能捕捉左右像素差异。

圆环内外轮廓同时出现，说明曲线边缘在连续扫描窗口中能够被逐点检测。

输出图仍为灰度强度图，而不是二值化图；亮度越高表示 $|G_x| + |G_y|$ 越大。

7.4 扩展波形与完成条件观察

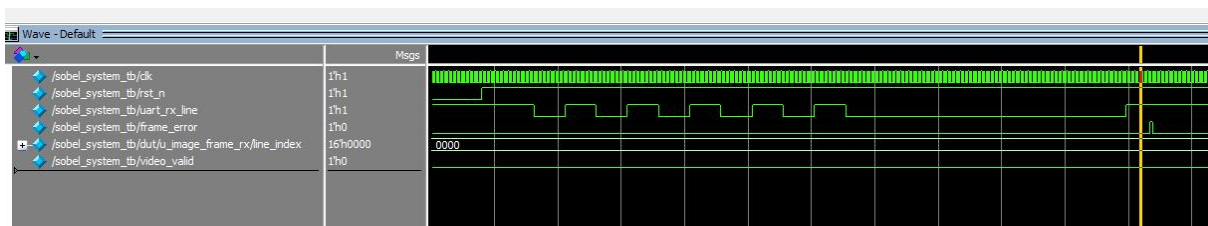


图 10 扩展仿真波形：frame_err、line_index、video_valid 等信号

扩展波形中可以继续观察到 `clk` 与 `rst_n` 的基础时序，`uart_rx_line` 表示仿真字节流输入过程；`line_index` 用于显示 `image_frame_rx` 当前解析到的行号；`video_valid` 表示后级视频像素流输出有效。截图中 `frame_err` 保持为低电平，说明帧头、行头、图像宽高和 RGB888 格式字段没有触发协议错误。

当输入图更换后，RTL 的控制流程没有变化：UART 字节先进入 `uart_rx`，再由 `image_frame_rx` 解析为 RGB 像素和坐标，随后进入 `rgb_to_gray` 与 `sobel_core`。只要帧格式合法、行号小于 `frame_height`、每行像素数达到 `frame_width`，系统即可按同样流程输出边缘图。因此扩展实验验证的是数据内容变化对 Sobel 输出的影响，而不是依赖某一张固定测试图才能通过仿真。

8. 结论

本实验完成了基于 UART 字节流的 RGB888 图像输入、帧协议解析、灰度转换、Sobel 边缘检测和视频像素流输出的 RTL 仿真验证。

仿真输出和波形共同表明：`rgb_valid`、`gray_valid` 与 `edge_valid` 按数据流顺序产生，Sobel 输出图能够突出输入图像中的主要边缘，`edge_frame_done/video_frame_done` 可作为整帧处理结束标志。

后续若迁移到 FPGA 开发板，可继续复用当前 UART 帧格式和 Sobel 核心，只需将 testbench 中的字节流模型替换为真实 UART 输入或上位机发送链路。